

CURSO DE STUDIO 5000 LOGIX DESIGNER

MÓDULO 3

Tags y tipos de datos

Capítulo técnico de estudio y referencia

Automatización Industrial • Allen-Bradley / Rockwell Automation • Studio 5000

Tag: Conveyor01_SpeedRef

Nombre	Scope	Tipo	Dato	Uso
Conveyor01_SpeedRef	Controller	Base	REAL	Referencia de velocidad

Ejemplo educativo; la nomenclatura real debe seguir el estándar de la planta.

Objetivo: comprender cómo Studio 5000 representa, organiza y manipula la información de una máquina mediante Tags y tipos de datos, y diseñarlos con criterio profesional.

Precisión: algunas capacidades dependen del controlador, firmware y versión de Studio 5000. Cuando una característica es específica, se indica.

1. Objetivos de aprendizaje

Al terminar podrás crear y analizar Tags, seleccionar tipos de datos, distinguir scopes, usar alias, arrays y UDT, y diagnosticar problemas de datos en una aplicación Logix.

- Reconocer la diferencia entre Tag, scope y tipo de dato.
- Seleccionar BOOL, SINT, INT, DINT, LINT y REAL según la aplicación.
- Entender estructuras, arrays y User-Defined Data Types (UDT).
- Distinguir Base, Alias, Produced y Consumed Tags.
- Diseñar nomenclatura y descripciones útiles para mantenimiento.
- Relacionar Tags con I/O, Ladder, HMI, variadores e instrumentación.
- Diagnosticar errores de rango, tipo, alias, scope y escalamiento.

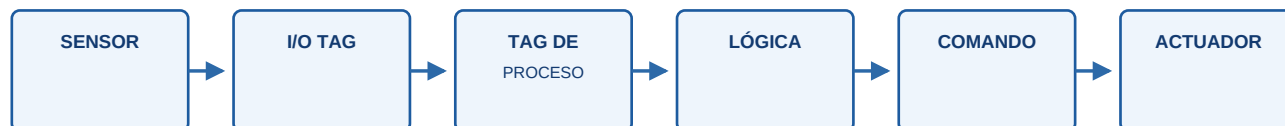
Ruta de aprendizaje



2. Introducción: pensar en datos, no solo en bits

En Logix, la lógica trabaja con nombres que representan datos: órdenes, estados, mediciones, parámetros, alarmas y estructuras de equipos. Rockwell define el Tag como el mecanismo básico para asignar memoria, referenciar datos desde la lógica y monitorearlos.

En planta, esto permite que el programa se parezca al proceso: **Motor01.RunCmd** comunica intención y facilita mantenimiento. Un diseño profesional busca que otro técnico pueda entender el dato sin depender del autor original.



Cadena mental: instrumento → módulo I/O → Tag → lógica → resultado → actuador/HMI/otro controlador.

3. ¿Qué es un Tag?

Definición: nombre asociado a un área de datos del controlador. El Tag identifica información que la lógica utiliza.

Cómo funciona: un Base Tag se crea con nombre, scope y tipo de dato. El tipo determina qué clase de información puede almacenar y cómo se interpreta.

¿Para qué sirve? estados, comandos, mediciones, parámetros, alarmas, contadores, temporizadores y estructuras.

Ejemplo sencillo: **Motor_RunCmd** como BOOL: 0 = no comandar; 1 = comandar.

Ejemplo industrial: una banda puede tener Motor01_StartCmd, Motor01_RunFb, Motor01_Faulted y Motor01_SpeedFbk.

Relación: la lógica puede leer/escribir Tags; Data Monitor puede observar valores; HMI y otros sistemas pueden acceder según configuración.

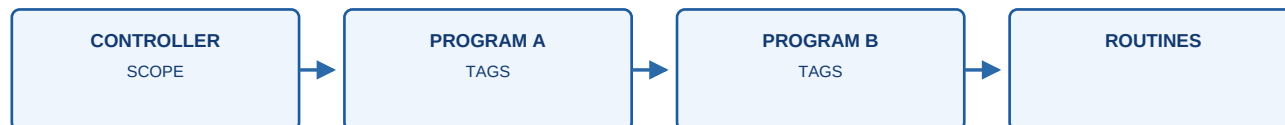
Buenas prácticas: nombres consistentes, descripción útil y una sola responsabilidad clara por Tag.

Error común: crear variables sin definir quién las escribe y quién las consume.

4. Scope: Controller vs Program

Controller Scope: el dato es accesible desde los programas del controlador. Es el equivalente conceptual a una variable global del controlador.

Program Scope: el dato se utiliza dentro del programa correspondiente. Es útil para encapsular información interna.



Ejemplo industrial: un estado común del sistema puede ser controller scope; una variable interna de una secuencia puede ser program scope.

Advertencia: si existe un Tag de controller scope y otro de program scope con el mismo nombre, la rutina utiliza el Tag de program scope. Esto debe revisarse durante troubleshooting.

Recomendación: mantén globales solo los datos que realmente deben compartirse y encapsula variables internas.

5. Tipos de datos atómicos

Los tipos atómicos son los bloques básicos con los que se construyen otros tipos. La documentación actual de Rockwell identifica BOOL, SINT, INT, DINT y REAL como tipos atómicos predefinidos; determinados controladores/versiones soportan además tipos extendidos como LINT y tipos sin signo.

Tipo	Uso típico	Rango / característica	Ejemplo
BOOL	Estado/comando	0 o 1	Start, Run, Fault
SINT	Entero pequeño	-128 a 127	Código pequeño
INT	Entero 16-bit	-32,768 a 32,767	Conteo limitado
DINT	Entero 32-bit	-2,147,483,648 a 2,147,483,647	Contadores
LINT	Entero 64-bit	Rango amplio; depende del controlador	Acumulación grande
REAL	Punto flotante	Valor decimal	Temperatura, presión, velocidad

Ingeniería: no elijas un tipo solo porque “acepta el valor”. Considera rango, precisión, cálculos, comunicación y compatibilidad con instrucciones/hardware.

6. Elegir correctamente el tipo de dato

BOOL: úsalo para estados lógicos. No conviertas un estado binario en REAL solo porque una HMI lo muestra.

DINT: suele ser una opción práctica para conteos y enteros de propósito general cuando no se necesita un rango mayor.

REAL: apropiado para magnitudes continuas o cálculos con decimales, como temperatura, presión, caudal o referencia de velocidad. Debe considerarse la precisión necesaria.

INT/SINT: útiles cuando el rango y la representación lo justifican; evita usarlos por costumbre sin revisar la aplicación.

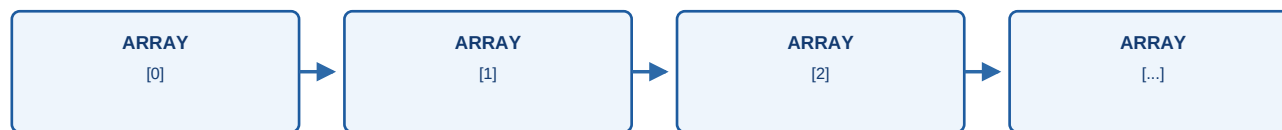
LINT y tipos extendidos: su disponibilidad y soporte dependen del controlador/versión; verifica la documentación del equipo antes de estandarizarlos.

Ejemplo de instrumentación: un transmisor 4–20 mA puede llegar como valor de entrada del módulo; el programa puede convertir/escalar ese dato a una variable de proceso, por ejemplo **Tank_Pressure_PV : REAL**. El tipo del Tag final debe corresponder a la operación que realizará el programa.

Error común: confundir el formato de visualización con el tipo de dato. Decimal, hexadecimal, binary, etc. son estilos de presentación; no cambian por sí mismos el tipo fundamental del Tag.

7. Arrays: múltiples elementos del mismo tipo

Un array agrupa varios elementos del mismo tipo de dato. Rockwell documenta arrays de una o varias dimensiones; los elementos de un array deben ser del mismo tipo atómico según la definición descrita para este caso.



Ejemplo: `Tank_Temp[10]` podría representar diez lecturas de temperatura si el proyecto está diseñado así.

Ejemplo industrial: una máquina con 8 estaciones puede usar un array de estados o resultados cuando las estaciones comparten exactamente la misma estructura de información.

Ventaja: permite recorrer datos con lógica repetitiva y evita crear decenas de Tags independientes cuando el proceso realmente es una colección.

Riesgo: un índice incorrecto puede apuntar a otro elemento. En troubleshooting, verifica siempre qué índice utiliza la lógica.

8. Estructuras y UDT

Una estructura contiene miembros de diferentes tipos. Un User-Defined Data Type permite crear una estructura propia que representa una máquina, equipo o entidad del proceso.

UDT: MOTOR_STATUS			
Cmd_Start	BOOL	Comando	Motor_01 MOTOR_STATUS Motor_01.SpeedFbk Motor_01.Faulted Motor_01.Running Una estructura representa una entidad del proceso.
Running	BOOL	Feedback	
Faulted	BOOL	Falla	
SpeedRef	REAL	Referencia	
SpeedFbk	REAL	Feedback	
Current	REAL	Corriente	

Ejemplo industrial: en vez de crear 10 Tags sueltos para cada motor, se puede crear un UDT MOTOR_STATUS y luego instanciar Motor01, Motor02, Motor03, etc. La estructura puede contener comandos, feedback, fallas y variables de proceso.

Buenas prácticas: diseña UDTs alrededor de entidades reales y estables; documenta cada miembro; evita meter datos sin relación solo para “tenerlos juntos”.

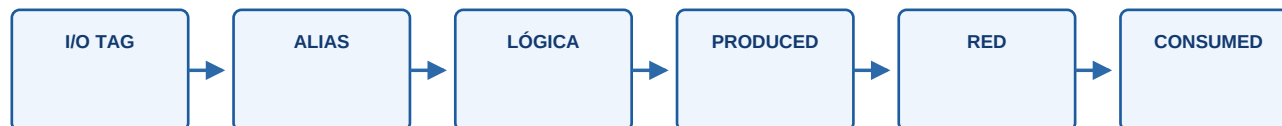
Ventaja de mantenimiento: la estructura crea un patrón repetible. Esto facilita AOI, HMI y diagnósticos cuando el proyecto está diseñado de forma consistente.

9. Base Tags, Alias Tags, Produced y Consumed

Base Tag: define dónde se almacena el dato. Es el tipo normal de Tag cuando no se especifica otra relación.

Alias Tag: referencia memoria definida por otro Tag. Puede apuntar a un Base Tag, otro alias, un miembro de estructura o un bit, según las reglas del entorno.

Produced Tag: controlador pone el dato a disposición de otros controladores. **Consumed Tag:** el valor proviene de un controlador remoto. Estos conceptos son relevantes en comunicación entre controladores y tienen restricciones específicas.

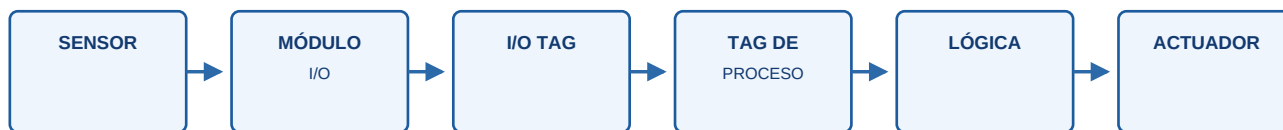


Ejemplo industrial: un alias puede dar un nombre funcional a una señal física; Produced/Consumed puede compartir datos entre controladores sin convertir el intercambio en una colección desordenada de mensajes.

Advertencia: no asumas que cualquier tipo o cualquier Tag puede producirse/consumirse. Rockwell establece restricciones de scope, tamaño y tipos; verifica la versión y controlador.

10. Tags de I/O y relación con hardware

Cuando se agrega un módulo I/O al proyecto, Studio 5000 crea Tags asociados al módulo. Rockwell documenta categorías de datos de configuración, entrada y salida en esos Tags.



Ejemplo: un sensor de proximidad llega a un canal digital. El Tag de I/O representa el estado del módulo; la lógica puede copiar o interpretar ese estado en un Tag funcional como **Conveyor01_ProductPresent**.

¿Por qué separar? Porque el Tag de proceso expresa intención de máquina y reduce dependencia de nombres físicos. Esta separación puede hacer el programa más legible y facilitar cambios de hardware.

Troubleshooting: si el Tag funcional no cambia, sigue la cadena hacia atrás: lógica → Tag de proceso → I/O Tag → módulo → cableado/sensor.

11. Tags dentro de Ladder

La lógica Ladder usa Tags como operandos de instrucciones. El tipo de dato debe ser compatible con la operación. Una salida BOOL no representa lo mismo que una referencia REAL.

Ejemplo conceptual de arranque:

```
Start_PB AND Permissive_OK AND NOT Motor_Faulted → Motor_RunCmd
```

Cada Tag tiene una responsabilidad: Start_PB representa la petición, Permissive_OK una condición de habilitación, Motor_Faulted una condición de bloqueo y Motor_RunCmd el resultado lógico.

Buenas prácticas: evita escribir el mismo comando desde muchas rutinas sin una razón clara. Define una fuente de verdad para comandos y estados.

Errores frecuentes: BOOL donde se necesita REAL; DINT usado como si fuera estado; Tag escrito por dos lugares; alias apuntando a una señal inesperada; nombre correcto pero scope incorrecto.

12. Tags, HMI y acceso externo

Los Tags pueden tener configuración de acceso externo. La documentación de Rockwell incluye opciones como Read/Write, Read Only o None para aplicaciones externas; la disponibilidad exacta depende del entorno y versión.

Principio profesional: no expongas Write a una HMI solo porque “funciona”. Un operador debe escribir únicamente aquello que el diseño permite modificar de forma segura.



Ejemplo: una referencia de velocidad puede ser escribible desde HMI si existe una estrategia de límites, permisos, modo Auto/Manual y validación. Un feedback de velocidad normalmente debería ser de solo lectura.

13. Ejemplo completo: motor con PowerFlex y protección

Objetivo: representar un motor de banda con comando, feedback, falla, referencia y corriente. El ejemplo es conceptual; los Tags reales del PowerFlex/E300/E3 Plus dependen del perfil, módulo, versión y configuración.

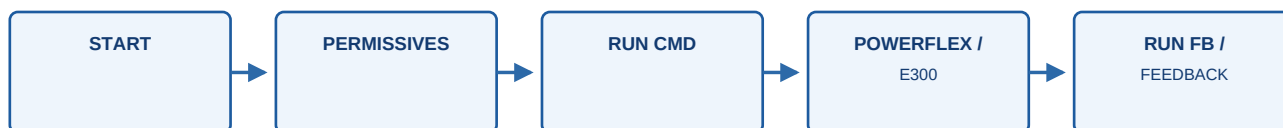
1) Diseñar datos

```
Motor01_RunCmd : BOOL
Motor01_RunFb : BOOL
Motor01_Faulted : BOOL
Motor01_SpeedRef : REAL
Motor01_SpeedFbk : REAL
Motor01_Current : REAL
```

2) Definir responsabilidades

RunCmd lo genera la lógica. RunFb proviene del drive/I/O. Faulted representa la condición de falla. SpeedRef es la referencia enviada. SpeedFbk y Current son feedbacks.

3) Flujo



4) Diagnóstico

Si RunCmd=0, diagnostica lógica/permisives. Si RunCmd=1 pero RunFb=0, pasa al drive, comunicación, enable/inhibit, fault y feedback. Si RunFb=1 pero el proceso no se mueve, investiga referencia, drive, motor y mecánica.

5) Evolución

Cuando el patrón se repite para muchos motores, un UDT puede agrupar los datos. Si el proyecto lo justifica, una AOI puede encapsular comportamiento, pero eso requiere estudiar AOIs como tema específico.

14. Troubleshooting de Tags y tipos de datos

La regla es aislar primero el tipo de problema: definición, scope, lógica, I/O, comunicación o dispositivo.

Síntoma	Posibles causas	Cómo comprobar	Solución recomendada
Tag no aparece	Scope equivocado / nombre distinto	Revisar Controller/Program Tags y búsqueda	Usar el Tag correcto; evitar duplicados ambiguos
Valor no cambia	Nadie escribe / condición falsa / I/O	Data Monitor + Cross Reference + I/O	Encontrar fuente de escritura
Tipo incompatible	BOOL/REAL/DINT incorrecto	Revisar Data Type y operandos	Cambiar diseño o convertir conscientemente
Valor fuera de rango	Tipo insuficiente / escala incorrecta	Revisar rango, raw/engineering	Corregir tipo o escalamiento
Alias inesperado	Target equivocado	Revisar Alias For / Base Tag	Corregir referencia y documentar
UDT difícil de mantener	Miembros mal definidos	Revisar estructura y usos	Rediseñar con estándar
HMI no lee/escribe	External Access / conexión	Revisar propiedades y HMI	Ajustar acceso según filosofía
Datos remotos no llegan	Produced/Consumed/configuración	Revisar conexión y estados	Verificar ambos controladores y compatibilidad

15. Buenas prácticas de ingeniería

Nomenclatura consistente

Usa prefijos/sufijos que expresen equipo y función. Ej.: Motor01_RunCmd, Motor01_RunFb.

Descripción siempre que aporte contexto

Documenta origen, propósito, unidades o condiciones importantes.

Una fuente de verdad

Define quién escribe cada comando/estado para evitar conflictos.

Scope intencional

Controller scope para datos realmente compartidos; Program scope para datos internos.

Tipos adecuados

Selecciona por rango, precisión, cálculo y compatibilidad, no por costumbre.

UDT para equipos repetitivos

Si 20 motores tienen la misma información, busca un modelo común.

Separar I/O de intención

Siempre que el diseño lo justifique, usa Tags funcionales que desacoplen lógica de hardware.

Cuidado con Online

No escribas valores ni fuerces Tags sin evaluar el impacto en la máquina.

Documentar cambios

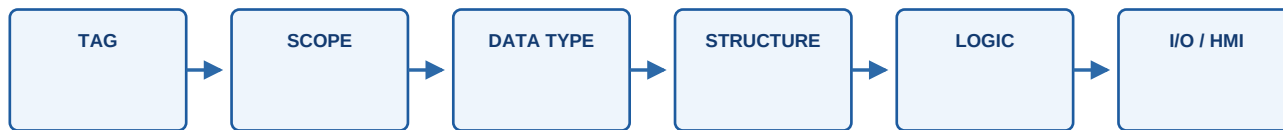
Registra qué Tag cambió, por qué y cómo se verificó.

No inventar compatibilidad

Verifica catálogo, firmware, versión de Studio 5000 y documentación del dispositivo.

Principio Senior: un buen sistema de Tags permite entender la máquina sin tener que “adivinar” qué quiso decir el programador.

16. Resumen técnico



Tag: nombre que representa datos del controlador.

Scope: determina dónde puede utilizarse el dato.

BOOL: estados y comandos binarios.

DINT: enteros de propósito general cuando el rango es suficiente.

REAL: magnitudes con decimales y cálculos de punto flotante.

Array: colección de elementos del mismo tipo.

UDT: estructura diseñada por el usuario para representar una entidad.

Alias: referencia a memoria de otro Tag.

Produced/Consumed: intercambio de Tags entre controladores bajo reglas específicas.

Troubleshooting: sigue el dato desde su fuente hasta su consumidor; no cambies hardware sin evidencia.

17. Ejercicios progresivos

Primero resuelve los ejercicios. Las soluciones están en la sección siguiente.

Nivel básico — 5 ejercicios

1. Define Tag, Scope y Data Type con un ejemplo industrial.
2. Indica qué tipo elegirías para StartCmd, SpeedRef y PartCount y justifica.
3. Explica la diferencia entre Controller Scope y Program Scope.
4. ¿Qué diferencia hay entre Base Tag y Alias Tag?
5. Explica por qué una descripción de Tag es importante para mantenimiento.

Nivel intermedio — 5 ejercicios

1. Diseña 6 Tags para un motor con comando, feedback, falla, velocidad y corriente.
2. Crea conceptualmente un array para 8 estaciones de una máquina y explica cuándo sería buena idea.
3. Un Tag HMI debe permitir cambiar una referencia de velocidad. ¿Qué controles de diseño considerarías?
4. Un Tag esperado no cambia. Diseña una secuencia de diagnóstico usando Data Monitor, Cross Reference e I/O.
5. Explica por qué un mismo nombre en Controller Scope y Program Scope puede causar confusión.

Nivel avanzado — 3 ejercicios

1. Diseña un UDT para una bomba con comando, estado, falla, presión, corriente y horas de operación.
2. Propón una arquitectura de Tags para 12 motores repetitivos y explica dónde usarías UDT/array.
3. Dos controladores deben compartir estados de una máquina. Explica qué revisarías antes de usar Produced/Consumed.

Troubleshooting

START_PB=1, Permissive_OK=1, Motor_Faulted=0, pero Motor_RunCmd permanece en 0. No cambies nada todavía. Indica qué Tags y referencias revisarías, en qué orden y qué resultado esperarías en cada paso.

18. Soluciones explicadas paso a paso

1. Tag = nombre de un dato; Scope = alcance; Data Type = representación del dato. Ej.: Motor01_RunCmd : BOOL.
2. StartCmd → BOOL; SpeedRef → REAL; PartCount → DINT, suponiendo un conteo dentro del rango disponible.
3. Controller Scope permite acceso desde programas del controlador; Program Scope limita el uso a las rutinas del programa.
4. Base Tag define el almacenamiento; Alias Tag referencia memoria de otro Tag.
5. La descripción reduce tiempo de interpretación y ayuda a mantenimiento, puesta en marcha y troubleshooting.
6. Ejemplo: Motor01_RunCmd BOOL; Motor01_RunFb BOOL; Motor01_Faulted BOOL; Motor01_SpeedRef REAL; Motor01_SpeedFbk REAL; Motor01_Current REAL.
7. Array es útil cuando las estaciones son homogéneas y se procesan con lógica repetitiva. Si cada estación es distinta, una estructura puede ser más clara.
8. Definir límites, modo de operación, permisos, validación, unidades, rango, fuente de verdad y condiciones de seguridad antes de permitir escritura desde HMI.
9. Data Monitor → confirmar valor; Cross Reference → identificar escritores/lectores; I/O → comprobar origen físico; luego lógica y comunicación.
10. Porque la rutina puede resolver el nombre hacia el Program Scope. Siempre confirma scope durante el diagnóstico.
11. UDT: Cmd_Start BOOL; Running BOOL; Faulted BOOL; Pressure REAL; Current REAL; RunHours DINT/otro tipo apropiado según requerimiento.
12. UDT para representar cada motor y un array de UDT si el proceso es realmente homogéneo; separar comandos, estados y datos de I/O según la arquitectura.
13. Revisar tipo de datos, scope, compatibilidad, configuración de ambos controladores, conexión, nombres, tamaños/estructuras y estados de comunicación.
14. Troubleshooting: 1) confirmar los cuatro Tags en Data Monitor; 2) localizar el rung de Motor_RunCmd; 3) comprobar si existe otra condición no considerada; 4) usar Cross Reference; 5) seguir hasta el origen; 6) solo después pasar a I/O/drive si el comando realmente se genera.

19. Evaluación final

No consultes las soluciones mientras la realizas.

Preguntas teóricas — 10

1. ¿Qué es un Tag?
2. ¿Qué es Scope?
3. ¿Cuándo usarías BOOL?
4. ¿Qué diferencia práctica existe entre DINT y REAL?
5. ¿Qué es un Array?
6. ¿Qué es un UDT?
7. ¿Qué es un Alias Tag?
8. ¿Qué diferencia hay entre Controller y Program Scope?
9. ¿Para qué sirve Data Monitor?
10. ¿Qué debes verificar antes de usar Produced/Consumed Tags?

Análisis — 5

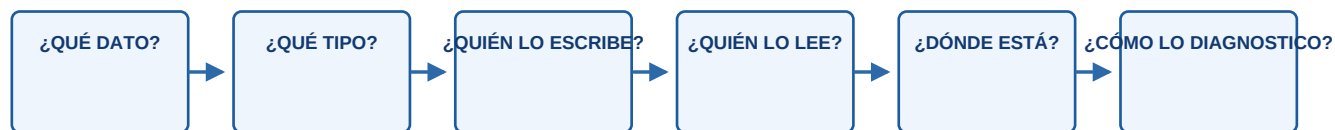
1. Un feedback de velocidad aparece como 0 aunque el drive está girando. ¿Qué capas analizarías?
2. Una HMI escribe un valor que la lógica cambia inmediatamente. ¿Cómo encontrarías quién escribe?
3. Un array tiene datos correctos excepto una estación. ¿Qué revisarías?
4. Dos programas usan Motor_StartCmd. ¿Cómo determinarías cuál Tag se está usando?
5. Un UDT cambia y aparecen errores en el proyecto. ¿Qué revisarías antes de modificarlo otra vez?

Problema práctico integrador

Diseña la estrategia de Tags para una banda con sensor de producto, motor PowerFlex, referencia de velocidad, feedback, falla de protección y HMI. Incluye nombres, tipos, scope, relación con I/O y una estrategia de diagnóstico.

Criterio de dominio: puedes diseñar Tags con intención, seleccionar tipos correctamente, explicar el scope y seguir un dato desde sensor/I/O hasta lógica, HMI o actuador.

20. Hoja de estudio para planta / entrevista



- Estado binario → normalmente BOOL.
- Entero → considerar DINT como opción general si el rango lo permite.
- Magnitud decimal → considerar REAL.
- Muchos elementos homogéneos → Array.
- Equipo repetitivo → UDT y, si aplica, array de UDT.
- Dato compartido → revisar Controller Scope.
- Dato interno → considerar Program Scope.
- Hardware físico → revisar I/O Tags y configuración del módulo.
- Falla de valor → Data Monitor + Cross Reference + I/O + lógica.
- Comunicación entre controladores → revisar Produced/Consumed y compatibilidad.
- Antes de editar Online → respaldo, autorización, estado de máquina y plan de reversión.

21. Referencias técnicas

Este capítulo fue elaborado tomando como base documentación oficial de Rockwell Automation para Studio 5000 Logix Designer y manuales de diseño de controladores Logix. La referencia exacta debe comprobarse contra la versión del proyecto en planta.

- Studio 5000 Logix Designer Online Help — Enter Tags and Aliases.
- Studio 5000 Logix Designer Online Help — Using User-Defined Data Types.
- Studio 5000 Logix Designer Online Help — Tag Editor and Data Monitor.
- Studio 5000 Logix Designer Online Help — Data Scope / Controller Scope.
- Studio 5000 Logix Designer Online Help — New Parameter or Tag dialog.
- Logix 5000 Controllers Design Considerations Reference Manual — tipos de datos y consideraciones de memoria.

Advertencia profesional: catálogo de controlador, firmware, versión de Studio 5000, módulos I/O y perfil del dispositivo pueden cambiar capacidades y restricciones. En una intervención real, verifica siempre la documentación específica del hardware.

Fin del capítulo — Tags y tipos de datos